Flutter Travel App Report

Lab 3 — Mobile Development Assigment

Team

1. Ange Thierry - 223012991
2. Lucide Pascale – 223018552

Course: Mobile Development

Department: Computer Engineering

Academic Year 2025–2026

1. Introduction

This report documents the design and development of a Flutter Travel App built as part of Lab 3. The application allows users to browse travel destinations, view detailed information about each location, and complete a booking flow. The report covers the app's screen layouts, widget usage, and navigation architecture.

Project Overview

Platform: Flutter / Dart
Screens: Home · Detail · Booking
Theme: Custom AppTheme (colors, gradients, typography)
Data: Hard-coded destinations in travel_data.dart

2. Screens & Layout Choices

2.1 Home Screen

The Home Screen is the app's entry point. It presents a curated list of destinations through a scrollable grid with a search bar and category filter chips at the top.

Layout Choices

- **Scaffold + AppBar:** A

Provides the standard app shell with a top bar containing the title and action icons.

- **CustomScrollView + SliverGrid:** A

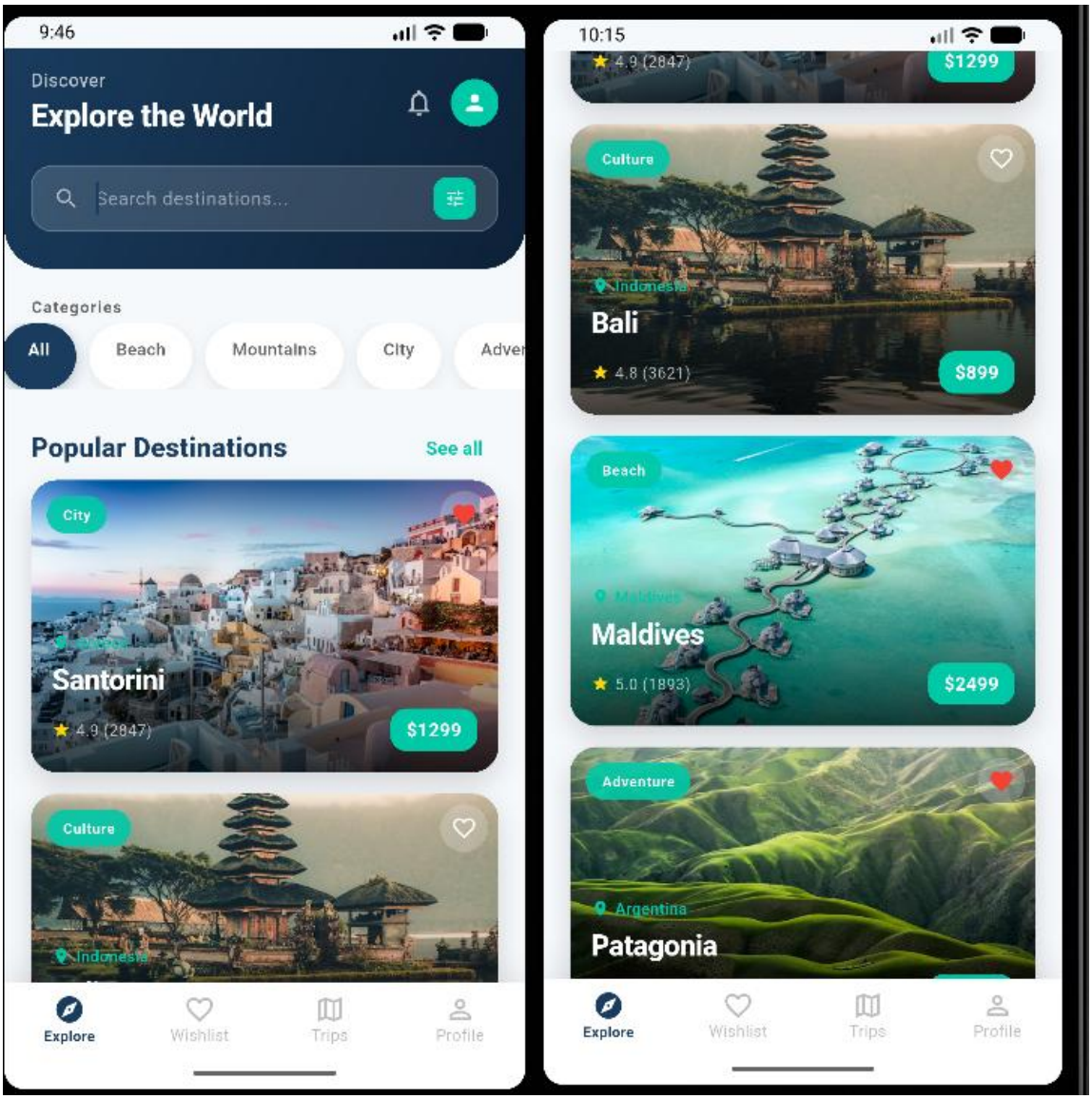
Enables a collapsible header and an efficient grid layout for destination cards, avoiding rebuilds on scroll.

- **SingleChildScrollView (horizontal) for category chips:** A

Lets users swipe through filter categories without taking up vertical space.

- **Stack inside DestinationCard:** A

Overlays the destination name and rating on top of the hero image using a gradient overlay.



2.2 Detail Screen

The Detail Screen displays in-depth information about a selected destination — a scrollable image carousel, description, amenities badges, and a prominent 'Book Now' call-to-action.

Layout Choices

- **PageView (image carousel):** A

Enables horizontal swiping between destination photos with dot indicator pagination.

- **SliverAppBar:** A

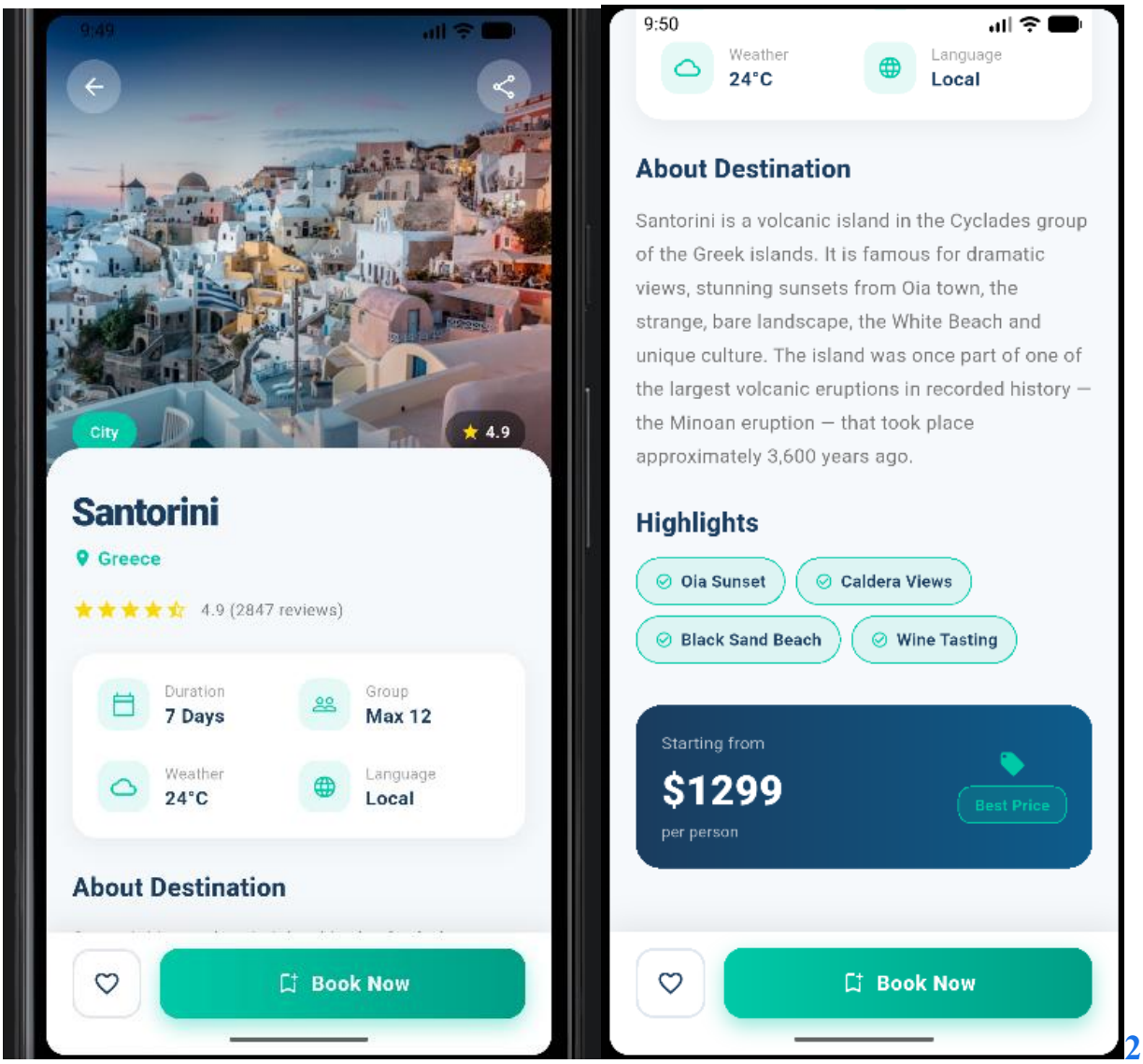
Collapses the hero image as the user scrolls down, keeping screen real estate optimised.

- **Wrap for amenity badges:** A

Auto-wraps AmenityBadge chips to the next line, accommodating any number of amenities responsively.

- **Expanded + Row for the bottom bar:** A

Pins the price and Book Now button at the bottom using a persistent BottomAppBar.



2.3

2.3 Booking Screen

The Booking Screen guides the user through selecting travel dates, number of passengers, and confirming the reservation. It ends with a confirmation dialog.

Layout Choices

- **Form + TextFormField:** A

Groups all input fields for built-in validation and easy submission via a GlobalKey<FormState>.

- **Column inside SingleChildScrollView:** A

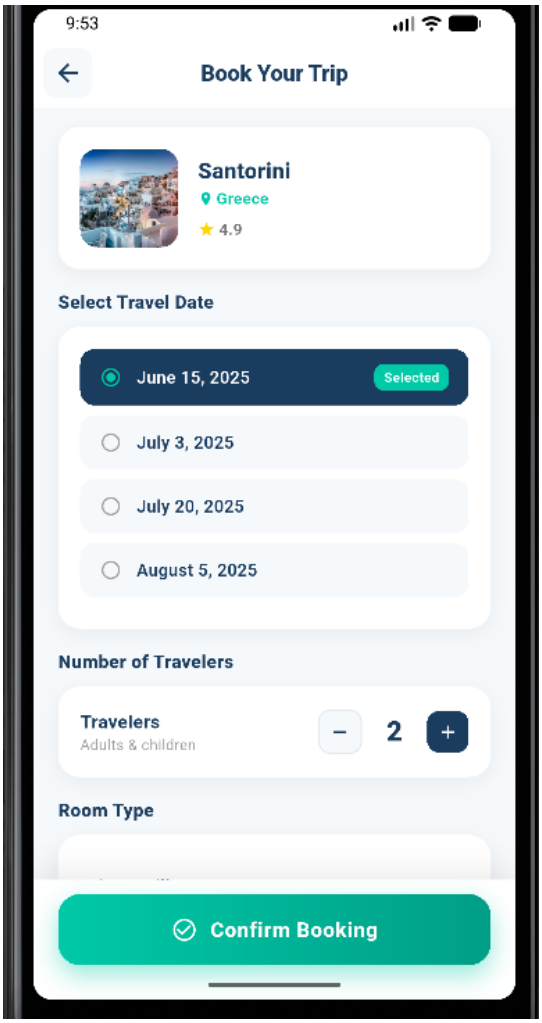
Allows the form to scroll on small screens without losing context.

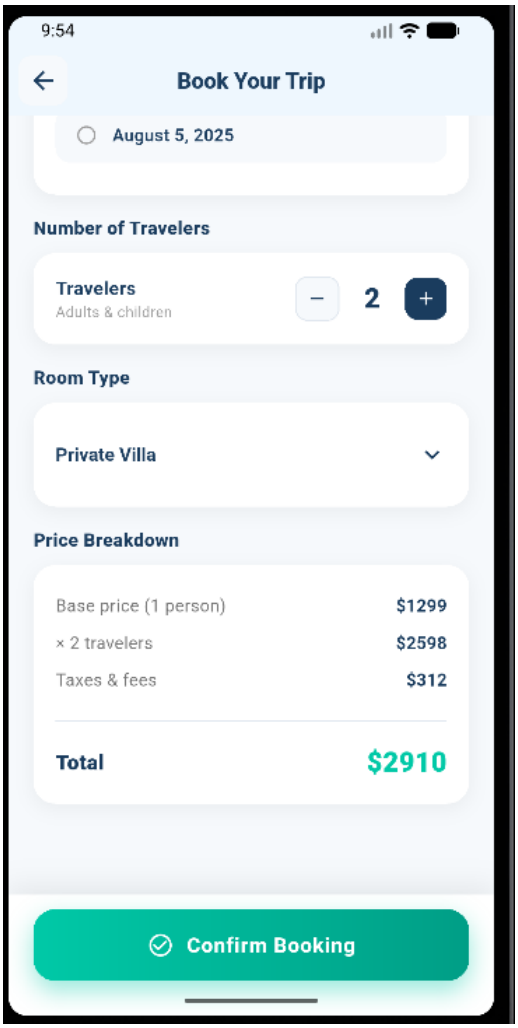
- **Card for price breakdown:** A

Visually separates the cost summary from the input fields, improving scannability.

- **AlertDialog (confirmation):** A

A modal overlay that confirms the booking before navigating back, preventing accidental submissions.





3. Widgets Used

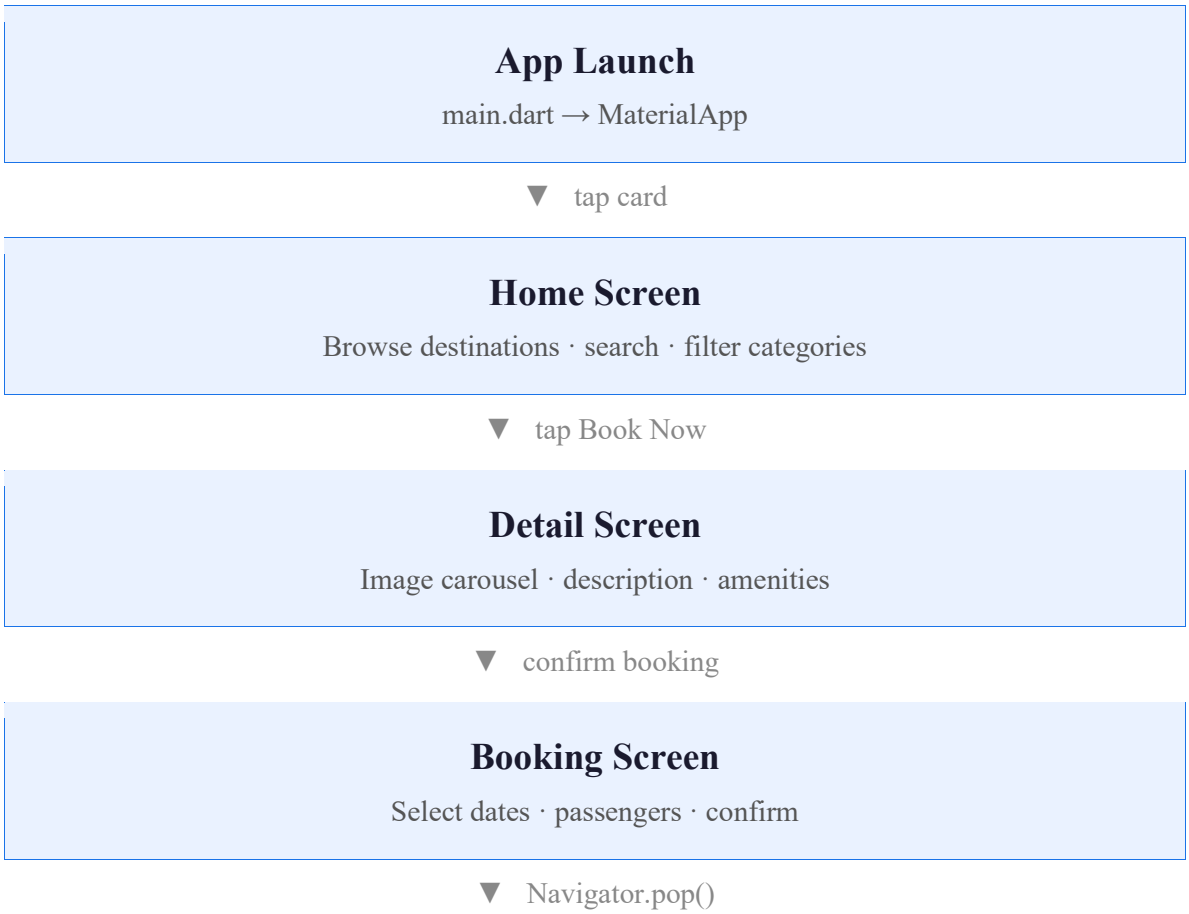
The table below lists all significant widgets used across the three screens, their category, and their specific role in the app.

Widget	Category	Purpose in the App
Scaffold	Layout	Base shell providing AppBar, body, and BottomAppBar for every screen
AppBar / SliverAppBar	Navigation	Top bar with title, back button, and action icons; collapses on Detail screen
CustomScrollView	Scrolling	Powers the sliver-based Home screen with collapsible header and grid
SliverGrid	Layout	Efficient grid of destination cards on the Home screen
PageView	Carousel	Horizontal image slider on the Detail screen with dot indicator
Stack	Layout	Overlays gradient + text on destination card images
Wrap	Layout	Flows AmenityBadge chips responsively on the Detail screen
Card	Material	Elevated container for price breakdown on Booking screen
Form + TextFormField	Input	Collects and validates date & passenger inputs on Booking screen
AlertDialog	Feedback	Modal confirmation dialog after booking submission
GestureDetector	Interaction	Detects taps on destination cards to navigate to Detail screen
AnimatedContainer	Animation	Smooth transition for active CategoryChip selection
ClipRRect	Styling	Rounds image corners on DestinationCard and ImageSlider
LinearGradient	Styling	Overlay gradient on card images for readable text contrast

Widget	Category	Purpose in the App
IconButton	Interaction	Favorite toggle inside DestinationCard
CircleAvatar	Display	Displays user avatar or destination icon in the AppBar
Divider	Layout	Visual separator between sections on Detail and Booking screens
SnackBar	Feedback	Temporary feedback message after actions (e.g., added to favorites)

4. Navigation Flow Diagram

The app uses Flutter's built-in Navigator with named routes and push/pop semantics. The diagram below shows the full navigation flow from app launch to booking confirmation.



Confirmation Dialog

AlertDialog → pop back to Home

4.1 Navigation Methods

Push (forward navigation)

```
onTap: () => Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => DetailScreen(destination: dest),
  ), // MaterialPageRoute
```

Push (detail → booking)

```
onTap: () => Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => BookingScreen(destination: destination),
  ), // MaterialPageRoute
```

Pop (back navigation)

booking_screen.dart → home (after confirm)

```
onTap: () => Navigator.pop(context),
```

4.2 Route Summary Table

From	To	Trigger
Home Screen	Detail Screen	Tap on a DestinationCard (GestureDetector)

Detail Screen	Booking Screen	Tap 'Book Now' button (ElevatedButton)
Booking Screen	Home Screen	Confirm dialog → Navigator.popUntil('/')
Any Screen	Previous Screen	Back button / Navigator.pop()

5. Conclusion

This Flutter Travel App demonstrates a clean, multi-screen architecture with a clear separation of concerns: data is centralised in travel_data.dart, theming is managed via app_theme.dart, reusable pieces live in widgets/, and each screen owns only its presentation logic.

The layout choices prioritise performance (SliverGrid, CustomScrollView) and visual polish (Stack overlays, gradients, ClipRRect) while keeping the codebase maintainable. Navigation follows Flutter's standard push/pop model, making the flow predictable and easy to extend with additional screens in future iterations.